Runtime Grounding: ECMAScript Semantics for the SKJS Value Domain

Deep Dive A for "Refinement and Inference Growth in SKJS"

Models revision `cf5a6bc7` of `main` (2026-07-09)

Abstract

The main soundness paper models the runtime as an abstract value domain Val equipped with predicates ($\approx$, truthy, $\mathsf{callable}$, $\mathsf{constructible}$, $\mathsf{isArrayV}$, P_{inst}); three of its ledger rows rest entirely on that domain: **L-TAG**, **L-INST**, and **L-ISARR**. This paper is the evidence behind their current table statuses. SKJS is a type checker only — there is no SKJS code generator, and checked programs run *unmodified* on a JavaScript engine — so each of these rows is really a claim about ECMAScript. This deep dive supplies the grounding: a realization map from the model's value domain onto ECMAScript language values under which the model's predicates are exactly the pullbacks of the spec's `IsStrictlyEqual`, `ToBoolean`, `typeof`, `IsCallable`/`IsConstructor`, and `IsArray`. Under it, **L-TAG** and **L-ISARR** are discharged (PROVED, modulo two named environment assumptions), and **L-INST** is decomposed into sharper named assumptions — two of which turn out to be VIOLATED, with checked-and-executed counterexamples produced while attempting the discharge. A third by-product refutes an implementation premise behind R-TYPEOF: class constructors implement $[[Call]]$, but the checker types them as definitely-not-callable. Section 9 gives the precise ledger edits for the main paper.

Contents

1 Purpose and Relation to the Main Paper **2**

2 Minimal Recap of the Modeled Runtime **2**

3 The Realization Map: Tags vs. ECMAScript Language Types **3**
3.1 The `typeof` table 4
3.2 Where the `void` tag fits 5

4 Strict Equality: $\approx$ is the Pullback of `IsStrictlyEqual` **5**

5 Truthiness: the Falsy Sets are `ToBoolean` **6**

6 Callable and Constructible **6**

7 `instanceof` and L-INST **7**
7.1 The spec algorithms 7
7.2 What the checker builds, and the gate 8
7.3 Conditional discharge and the honest caveats 8

8 `Array.isArray` and L-ISARR **10**
8.1 The spec side 10
8.2 The model side: how `ReadonlyArray` acquires the brand 11
8.3 Discharge 11

9 Status and Ledger Edits **12**

A Implementation and Specification Anchors **14**

B Witness Protocol **14**

1 Purpose and Relation to the Main Paper

This document is deep dive A of the series supporting *Refinement and Inference Growth in SKJS: a Soundness Model of the Type Checker* (the *main paper*), pinned like it to revision `cf5a6bc7`. Its job is to discharge, sharpen, or refute the rows of the main paper's assumption ledger (its Section "Assumptions and Proof Obligations") that concern the *runtime* rather than the checker:

- **L-TAG** — every value carries exactly one tag, consistently with its static tag. *SKJS emits no code*: a checked program is executed, byte for byte, by an ordinary JavaScript engine, so the runtime counterpart of the model's Lemmas A0/A1 is a statement about ECMAScript's language types, provable from the specification (Section 3) — which is why the ledger anchors this row to ECMAScript rather than to any SKJS component.
- **L-INST** — $P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type built at an `instanceof` narrowing site (Section 7).
- **L-ISARR** — $\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$ (Section 8).

In addition it grounds two definitions the main paper introduces without external justification and then uses everywhere: the strict-equality relation $\approx$ (its Definition "Strict Equality"; used by R-Eq, R-EqField, and constant removal) and the per-tag truthiness table (its Definition "Truthiness"; used by R-Truthy/R-TruthyField and the residue lemma). Both are shown to be *exactly* the ECMAScript algorithms, transported along the realization map (Sections 4 and 5).

Failed discharge attempts are reported as numbered *Findings*. Three of them come with programs that the pinned checker accepts (verified by running `skjs check` at the pinned revision) and that misbehave when executed on Node: they are witnesses, not conjectures. All spec citations are to ECMA-262 as of the 2025 edition; clause *numbers* drift between editions, so each citation also names the clause.

Notation. We reuse the main paper's notation unchanged: Val, Tags, rtag, payloads $(\mathsf{Payload}_k)_k$, ObjVal, the object interface (props, get, has, $\mathsf{callable}$, $\mathsf{constructible}$, $\mathsf{isArrayV}$), $\approx$, truthy, and $[\![\cdot]\!]$. Implementation anchors (impl: `...`) point into revision `cf5a6bc7`.

2 Minimal Recap of the Modeled Runtime

From the main paper's Section 2 (Runtime Model): Tags is the nine-element set

$$\mathsf{Tags} = \{\mathsf{undef}, \mathsf{void}, \mathsf{null}, \mathsf{bool}, \mathsf{num}, \mathsf{str}, \mathsf{bigint}, \mathsf{sym}, \mathsf{obj}\},$$

and $\mathsf{Val} = \{(k, p) \mid k \in \mathsf{Tags}, p \in \mathsf{Payload}_k\}$ with $\mathsf{Payload}_{\mathsf{num}} = \mathbb{F}$ (IEEE-754 doubles including -0 and NaN), $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$, and $\mathsf{Payload}_{\mathsf{undef}} = \mathsf{Payload}_{\mathsf{void}} = \{\star\}$. The void tag is declared there to be "a checker fiction for the runtime value `undefined`"; this paper makes that statement precise. The relation $\approx$ is a partial equivalence (NaN irreflexive, $-0 \approx 0$, the $\mathsf{undef}/\mathsf{void}$ pair identified); truthy is given by per-tag falsy sets; objects come with the abstract interface above, read as a snapshot (**L-PURE**).

The three refinement rules whose soundness proofs consume the assumed rows are R-Typeof (via the $P_{\mathsf{typeof}=k}$ sets and the $\mathsf{constructible} \implies \mathsf{callable}$ interface fact), R-Inst (via **L-INST**), and R-IsArray (via **L-ISARR**).

3 The Realization Map: Tags vs. ECMAScript Language Types

ECMAScript defines exactly eight *language types*: Undefined, Null, Boolean, String, Symbol, Number, BigInt, and Object (ECMA-262 §6.1, "ECMAScript Language Types"). Every language value belongs to exactly one of them — the types are introduced as a partition of the value space, not as overlapping classifications. Write ESVal for the set of language values and $\mathsf{estag}(v)$ for the language type of $v \in \mathsf{ESVal}$.

Definition 1 (Realization Map)**.** $\gamma : \mathsf{Val} \to \mathsf{ESVal}$ *sends a model value to the language value it stands for:*

- $\gamma(\mathsf{undef}, \star) = \gamma(\mathsf{void}, \star) =$ ***`undefined`*** *and* $\gamma(\mathsf{null}, \star) =$ ***`null`****;*
- *on* bool*,* num*,* str*,* bigint*: the evident payload realization — the model chose its payload sets to* be *the spec's value sets (*$\mathbb{F}$ *is exactly the Number type, "double-precision 64-bit binary format IEEE 754-2019" including* -0 *and a single* NaN*, ECMA-262 §6.1.6.1;* $\mathbb{Z}$ *is exactly the BigInt type, ECMA-262 §6.1.6.2;* String *realizes as UTF-16 code unit sequences, ECMA-262 §6.1.4);*
- *on* sym *and* obj*: a fixed bijection between the abstract payload sets* SymVal*,* $\mathsf{Payload}'_{\mathsf{obj}}$ *and the runtime symbol/object identities of the executing program.*

Theorem 2 (Grounding of Tags; discharges **L-TAG**)**.** γ *is surjective onto the language values reachable by the program, and:*

(G1) $\mathsf{estag}(\gamma(v))$ *is determined by* $\mathsf{rtag}(v)$*, via the evident bijection* $\mathsf{Tags} \setminus \{\mathsf{void}\} \leftrightarrow \{$*Undefined, Null, Boolean, Number, String, BigInt, Symbol, Object*$\}$*, extended by* $\mathsf{void} \mapsto$ *Undefined.*

(G2) γ *is injective except on the single fiber* $\{(\mathsf{undef}, \star), (\mathsf{void}, \star)\}$*, whose two elements realize to the same value* ***`undefined`****.*

(G3) Distinct tags other than the pair $\{\mathsf{undef}, \mathsf{void}\}$ *realize to disjoint sets of language values.*

Consequently the runtime counterparts of the main paper's Lemmas A0 (tag consistency) and A1 (tag disjointness) hold: a language value inhabits denotations of at most one tag, except that ***`undefined`*** *inhabits both* undef*-* *and* void*-tagged denotations.*

Proof. (G1) and (G2) are immediate from Definition 1. (G3) follows from (G1)+(G2) because the eight language types partition ESVal (ECMA-262 §6.1). Surjectivity: every language type is covered, and the payload realizations are onto their value sets (bijections or identity). □

The residual obligation created by (G2) is that no rule of the checker may depend on *runtime-distinguishing* the tags undef and void — they are indistinguishable after realization. The main paper already maintains this discipline explicitly: R-EqUndef and the `typeof "undefined"` case of R-Typeof treat the two tags jointly; the truthiness residue keeps void on both branches; the one corner where a single tag of the pair could be consumed alone is closed there by the checker's rejection of `void`-typed strict-equality operands. Nothing further is needed here.

Anchoring. There is no code generator anywhere in the pinned revision; the runtime is whatever engine executes the unmodified source, so the ledger's L-TAG row is anchored to this theorem (equivalently, to ECMA-262 §6.1), not to any SKJS component. Note the row's phrase "consistent with its static tag" asks, at full strength, that checked expressions inhabit their checked types; that is **L-TC**, a different row, and this paper deliberately does not fold it back in.

3.1 The `typeof` table

The `typeof` operator (ECMA-262 §13.5.3, "The `typeof` Operator", Table "typeof Operator Results") maps every language value to a string as follows, where *callable* means "implements the $[[Call]]$ internal method":

Language type of v	`typeof` v	model tag(s)
Undefined	`"undefined"`	undef, void
Null	`"object"`	null
Boolean	`"boolean"`	bool
Number	`"number"`	num
String	`"string"`	str
BigInt	`"bigint"`	bigint
Symbol	`"symbol"`	sym
Object, not callable	`"object"`	obj
Object, callable	`"function"`	obj

Two quirks are load-bearing for the checker and both are already encoded in the main paper's $P_{\mathsf{typeof}=k}$ sets (its Definition "Evaluation of Condition Forms"): `typeof null === "object"` (the `null` row above; the model includes $[\![\texttt{null}]\!]$ in $P_{\mathsf{typeof}=\texttt{"object"}}$), and the callable/non-callable split of the Object type (the model's $P_{\mathsf{typeof}=\texttt{"function"}} = \{o \in \mathsf{ObjVal} \mid \mathsf{callable}(o)\}$). Granting Definition 6 below (callable = implements $[[Call]]$), the pullback along γ of "`typeof` $x = k$" is *exactly* $P_{\mathsf{typeof}=k}$ for all eight operands. So the evaluation semantics of R-Typeof is grounded; the soundness of its narrowing given that semantics is the main paper's Lemma "R-Typeof is Branch-Sound" and is not re-proved here. (Whether the checker's *types* populate the callable split correctly is a separate question — see Finding 8.)

Named environment caveat (A-DDA). Annex B of the spec (ECMA-262 §B.3.6, "The $[[IsHTMLDDA]]$ Internal Slot") sanctions host objects — in practice the single browser object `document.all` — for which `typeof` returns `"undefined"` and `ToBoolean` returns false despite being objects. Such a value would break rows one and eight of the table and the truthiness grounding below. We record **A-DDA**: *no* $[[IsHTMLDDA]]$ *object flows into checked code.* This holds in

any non-browser embedding (Node has none), and the pinned import stubs describe a Node-style environment (`process`, `fetch`/undici — no DOM).
impl: `import-stubs/@types/skjs/index.d.ts`

3.2 Where the `void` tag fits

The checker distinguishes `void` from `undefined` to restrict *programs* (a `void`-typed result should not be consumed), not to describe a runtime distinction; Theorem 2 (G2) is the formal statement that no runtime distinction exists. The grounded reading of every void-related clause in the main paper is obtained by applying γ: e.g. the cross-tag arm of $\approx$ (next section) is not a modeling convenience but a *theorem forced by* $\gamma(\mathsf{undef}, \star) = \gamma(\mathsf{void}, \star)$.

4 Strict Equality: $\approx$ is the Pullback of `IsStrictlyEqual`

The spec's strict-equality algorithm (ECMA-262 §7.2.16, $\texttt{IsStrictlyEqual}(x, y)$; the semantics of `===` via ECMA-262 §13.11) is: (1) if $\mathsf{estag}(x) \neq \mathsf{estag}(y)$, return false; (2) if x is a Number, return $\texttt{Number::equal}(x, y)$ — which returns false if either argument is NaN, true if both are $+0/-0$ in any combination, and value equality otherwise (ECMA-262 §6.1.6.1.13); (3) otherwise return $\texttt{SameValueNonNumber}(x, y)$ — payload equality per type: trivially true for Undefined and Null, mathematical-value equality for BigInt, code-unit equality for String, identity for Boolean/Symbol/Object (ECMA-262 §7.2.12).

Theorem 3 (Grounding of $\approx$)**.** *For all* $v, w \in \mathsf{Val}$*:* $v \approx w$ *iff* $\mathit{IsStrictlyEqual}(\gamma(v), \gamma(w)) = \mathit{true}$*.*

Proof. Case on the tag pair. If $\mathsf{rtag}(v), \mathsf{rtag}(w) \in \{\mathsf{undef}, \mathsf{void}\}$: both sides realize to `undefined`; step (1) passes, step (3) returns true; and $v \approx w$ by the first arm of the definition — both sides hold. If the tags are equal and not in the pair: realization preserves the language type, step (1) passes, and steps (2)/(3) compute exactly the payload condition of $\approx$'s second and third arms (NaN-irreflexivity and $-0 = 0$ from `Number::equal`; payload/identity equality from `SameValueNonNumber` — for BigInt this is mathematical equality and $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$ has no signed zero, so no analogue of the -0 clause arises). If the tags differ and are not the $\{\mathsf{undef}, \mathsf{void}\}$ pair: by Theorem 2 (G3) the realizations have different language types, step (1) returns false, and $\approx$ does not relate v, w — both sides fail. □

Every property of $\approx$ that the main paper proves or uses — the partial-equivalence lemma, $\mathsf{NaN} \not\approx \mathsf{NaN}$, the identification of the undef/void pair — is therefore inherited from the spec rather than stipulated.

Contrast: `SameValue` and `Object.is`. The spec has two finer equalities: `SameValue` (ECMA-262 §7.2.10; surfaced as `Object.is`) makes NaN reflexive and *separates* $+0$ from -0; `SameValueZero` (ECMA-262 §7.2.11; used by `Array.prototype.includes`, `Map`/`Set` keying) makes NaN reflexive and identifies the zeros. Neither is $\approx$. This matters in two directions: (i) the checker narrows only on `===`/`!==`, so only `IsStrictlyEqual` needs grounding — `Object.is` is not even in the pinned stubs, and `Map`/`Set`/`includes` are opaque signatures (a subtyping-surface concern); (ii) inside `OrdinaryHasInstance` the spec compares prototype-chain entries with `SameValue`, but on Objects `SameValue` and `IsStrictlyEqual` coincide (both are identity), so no discrepancy reaches Section 7.

Corollary 4 (The literal-type closure choice is forced)**.** *The main paper defines literal-type denotations as same-tag $\approx$-closures, so the numeric literal type* `0` *denotes* $\{+0, -0\}$*. Under Theorem 3 this is the unique sound choice for* R-Eq*'s false branch:* `x !== 0` *does not exclude* $x = -0$ *at runtime (*-0 `===` `0` *is* `true`*), so a denotation of* `0` *omitting* -0 *would let constant removal subtract a value still possible. It also matches* `tsc`*, which has no* -0 *literal type:* `const x: 0 = -0` *is accepted there, i.e.* `tsc`*'s literal type* `0` *likewise contains both zeros.*

5 Truthiness: the Falsy Sets are `ToBoolean`

`ToBoolean` (ECMA-262 §7.1.2) is defined by a per-type table: `undefined` and `null` $\mapsto$ false; Boolean $\mapsto$ itself; Number $\mapsto$ false iff the value is $+0$, -0, or NaN; String $\mapsto$ false iff empty; BigInt $\mapsto$ false iff 0; Symbol $\mapsto$ true; Object $\mapsto$ true (with the Annex-B $[[IsHTMLDDA]]$ exception covered by **A-DDA**).

Theorem 5 (Grounding of Truthiness)**.** *Under* ***A-DDA****, for all* $v \in \mathsf{Val}$*:*

$$\mathsf{truthy}(v) = \mathit{ToBoolean}(\gamma(v)).$$

Proof. Row-by-row comparison of the main paper's Definition "Truthiness" with the table above, through the tag correspondence of Theorem 2: the undef, void rows both realize to `undefined` (falsy — the model marks both tags falsy, consistently); the numeric falsy set $\{0, -0, \mathsf{NaN}\}$, the empty string, the zero bigint, and the always-truthy symbol/object rows are verbatim. The comparison is exact and complete: neither table has a row the other lacks. □

This grounds the predicate. Note where the burden then sits: the residue lemma of the main paper (its Lemma "Residues are Sound") additionally needs the *implementation's lexeme-level* falsiness test to agree with truthy on literal spellings — a checker-internal (not runtime) lexeme-level matter, violated for string literals whose escapes are stored undecoded, and out of scope here. The runtime side contributes no further caveat.

impl: `TType.sk / TPrimValueKind::isFalsy, SType::falsiness`

6 Callable and Constructible

Definition 6 (Grounded predicates)**.** *For* $o \in \mathsf{ObjVal}$*:* $\mathsf{callable}(o)$ *iff* $\gamma(o)$ *implements the* $[[Call]]$ *internal method (the spec's* `IsCallable`*, ECMA-262 §7.2.3);* $\mathsf{constructible}(o)$ *iff* $\gamma(o)$ *implements* $[[Construct]]$ *(*`IsConstructor`*, ECMA-262 §7.2.4).*

"Callable" therefore means *has the internal method*, not "can be invoked without throwing". This distinction is exactly what the `typeof` table keys on, and it is where class constructors live: a class constructor is an ECMAScript function object whose $[[Call]]$ *exists but throws* — step "If $F.[[IsClassConstructor]]$ is true, throw a `TypeError`" in the $[[Call]]$ of ECMAScript function objects (ECMA-262 §10.2.1). Hence `typeof (class {}) = "function"`.

Theorem 7 (Interface fact: constructible $\implies$ callable)**.** *For every* $o \in \mathsf{ObjVal}$*,* $\mathsf{constructible}(o)$ *implies* $\mathsf{callable}(o)$*.*

Proof. The spec states it directly in the definition of the Object type (ECMA-262 §6.1.7.2, "Object Internal Methods and Internal Slots"): "every object that supports $[[Construct]]$ must support $[[Call]]$; that is, every constructor must be a function object" (reiterated among the invariants of

the essential internal methods, ECMA-262 §6.1.7.3). It is also easy to audit exhaustively across the function kinds the spec defines: *ordinary functions* — [[*Construct*]] is installed only on function objects (arrow functions and methods get [[*Call*]] only); *class constructors* — both present, [[*Call*]] throwing (ECMA-262 §10.2.1); *bound functions* (ECMA-262 §10.4.1) — always have [[*Call*]], and [[*Construct*]] only if the bound target has it; *Proxy exotic objects* (ECMA-262 §10.5.12–§10.5.13) — a proxy has [[*Call*]] iff its target does and [[*Construct*]] iff its target does, so the implication lifts through proxies by induction on the target chain. □

This discharges the "JS fact" quoted in the main paper's Object Interface (its Definition "Object Interface") and used in the `"object"` case of its Lemma "R-TYPEOF is Branch-Sound": status PROVED.

Finding 8 (Class constructors refute the checker's call = None typing; unsound `typeof` narrowing)**.** *Attempting to ground* R-TYPEOF*'s object-branch treatment end-to-end turned up a genuine unsoundness. The checker's static-side object type for a class C (the type of the constructor* value*) is built with a construct signature but* no *call signature —* new = Some(…), call = None *(`TEnv::newStaticObject` passes `new` and omits `call`). Under the main paper's object semantics,* call = None *asserts* ¬callable *of every member; but by Definition 6 and ECMA-262 §10.2.1 the runtime constructor value* is *callable (`typeof C === "function"`). So* ⟦`typeof C`⟧ *excludes the value C itself —* ***L-TC*** *is violated at class-constructor values — and* R-TYPEOF*, whose narrowing is sound* given *membership, misfires. Checked witness (accepted by the pinned checker, exit 0;* `tsc` *rejects it):*

```
class C { a: number = 1 }
function g(): string {
  const F = C;
  if (typeof F === "function") {
    const s: string = F;   // accepted: the branch narrowed F to bottom
    return s;
  }
  return "not a function";
}
console.log(g());          // Node prints: [class C]
```

At runtime the true branch is taken; the checker had dropped the sole object member of F's type (call mode None → *drop in the `"function"`/true arm), leaving an uninhabited type from which the `string` annotation is vacuously reachable. g() returns a class constructor as a `string`. The false branch of `typeof F === "object"` misfires symmetrically. The defect is not in* R-TYPEOF*'s rule (the main paper's proof is correct given membership) nor in the runtime facts (grounded above); it is in the static-side type:* call = None *("definitely not callable") should be the maybe-callable mode* (true, σ)*, or the `"function"` arm should treat* new ≠ None *as callable, per Theorem 7. Proposed ledger row:* ***A-CTOR-CALL****, status* VIOLATED *(Section 9); a pinned test is recommended.*
impl: `TEnv.sk / newStaticObject (call omitted); TypeCheckRefine.sk / evalTypeofEquality`

7 `instanceof` and L-INST

7.1 The spec algorithms

Evaluation of `V instanceof T` calls `InstanceofOperator`(V, T) (ECMA-262 §13.10.2):

1. if T is not an Object, throw;
2. let h = `GetMethod`$(T,$ `@@hasInstance`$)$; if h is not `undefined`, call h with receiver T and argument V, and return the `ToBoolean` of the result;
3. otherwise, if T is not callable, throw; return `OrdinaryHasInstance`(T, V).

The default `@@hasInstance` method, on `Function.prototype` (ECMA-262 §20.2.3.6), exists on every function and simply runs `OrdinaryHasInstance`; it is non-writable and non-configurable, but a class can *shadow* it with a static method, and any object can carry an own `@@hasInstance`. `OrdinaryHasInstance`(C, O) (ECMA-262 §7.3.22): if C is a bound function, recurse on its target; if O is not an Object, return false; let P = `Get`$(C,$ `"prototype"`$)$ (throw if not an Object); walk O's $[[GetPrototypeOf]]$ chain, returning true iff some entry is `SameValue`-identical to P.

Ground $P_{\mathsf{inst}}(C) \triangleq \{v \in \mathsf{Val} \mid$ `InstanceofOperator`$(\gamma(v), \gamma(C)) = \mathsf{true}\}$; the throwing cases satisfy neither branch relation (main paper, Definition "Evaluation of Condition Forms"), so they are vacuous for narrowing. That $P_{\mathsf{inst}}(C) \subseteq \mathsf{ObjVal}$ is immediate from step 3 of `OrdinaryHasInstance` — *provided* evaluation reaches it, which step 2 does not guarantee. This is the first caveat.

7.2 What the checker builds, and the gate

R-Inst fires only on the true branch, and only when the RHS is a bare name that resolves to a class or interface *definition* (`hasClassDef`); a merely Function-typed RHS narrows nothing. The instance type T_C is built by `newObject` with type arguments from `makeTArgs` — verified at the pin: `makeTArgs` generates one *fresh ordinary inference variable* (`Subst::gen`, a candidate `TVID`) per class parameter and attaches exactly the parameter's *declared constraint*, if any (`tvar.setConstraint`); `Array` is special-cased to the single argument $\top$.

impl: `TypeCheckRefine.sk / evalCond (InstanceOf arm), evalInstanceof`

impl: `TEnv.sk / hasClassDef, makeTArgs, newObject`

This pins the accurate discharge premise, recorded in the main paper's L-INST row: the chosen arguments are fresh, but *not* unconstrained when the class declares parameter bounds (`class C<T extends U>`). Freshness plus declared-bounds-only is what actually holds — and declared bounds are harmless for the inclusion, because every runtime instance was constructed through a `new`-check that enforced them (under **L-TC** at construction sites).

7.3 Conditional discharge and the honest caveats

Theorem 9 (Conditional discharge of **L-INST**)**.** *Assume, for the class C at a narrowing site:*

- ***A-HASINST**: no `@@hasInstance` method other than the `Function.prototype` default is found on $\gamma(C)$ (step 2 resolves to the default);*
- ***A-PROTO**: no prototype surgery — objects with C.`prototype` in their chain were produced by `new` on C or a subclass (no `Object.create`, `Object.setPrototypeOf`, `Reflect`, or Proxy $[[GetPrototypeOf]]$ traps reachable), and C.`prototype` itself is unmodified;*
- ***L-TC** at construction: every `new`-site establishes the instance object type of the constructed class (fields present and typed, per the class-checking discipline).*

Then $P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]_{\rho'}$ at the post-instantiation state ρ', reading the fresh unbound arguments' denotations as unconstrained-within-declared-bounds.

Proof sketch. By **A-HASINST**, membership in $P_{\mathsf{inst}}(C)$ is decided by `OrdinaryHasInstance`, so v is an object with C.`prototype` in its chain. By **A-PROTO**, v was constructed by C or a subclass, as some instantiation $C\langle\vec{U}\rangle$ with $\vec{U}$ within the declared bounds. By **L-TC** at the construction site, v then satisfies the instance object type of $C\langle\vec{U}\rangle$: every declared field present with a $\vec{U}$-instantiated get-type, inherited fields included (subclass constructors must complete `super()`). Since T_C's arguments are *fresh and unbound at* ρ', each field constraint of T_C weakens to its declared bound, which $\vec{U}$ satisfies; so $v \in [\![T_C]\!]_{\rho'}$. Mutation after construction is governed by **L-MUT** exactly as for every other membership statement in the main paper. □

Each hypothesis must now be audited against the implementation. The audit *fails* for the first, and the aftermath of the theorem fails for the freshness clause; both failures are pinned.

Finding 10 (**A-HASINST** is violated: SKJS accepts `@@hasInstance` overrides and narrows anyway)**.** *The narrowing gate checks only that the RHS name has a class definition (`hasClassDef`); nothing rejects or inspects a static `[Symbol.hasInstance]` member, which SKJS class bodies* can *declare — computed symbol-keyed members resolve into symbol fields (the `FNComputed` arm of `addFields`), and `Symbol.hasInstance` is a stubbed `unique symbol`. The implementation itself documents the gap ("Mirrors tsc's TS2359 acceptance (modulo `Symbol.hasInstance`, which SKJS does not model)", `isInstanceofRhs`). Checked witness (accepted at the pin, exit 0):*

```
class C {
  a: number = 1;
  static [Symbol.hasInstance](x: unknown): boolean { return true; }
}
function f(x: unknown): number {
  if (x instanceof C) { return x.a; }   // narrowed to C
  return 0;
}
console.log(f("not a C"));              // Node prints: undefined
```

At runtime step 2 of `InstanceofOperator` finds the static method, every value is "an instance", and `f` returns `undefined` at type `number`. Two mitigations narrow the blast radius: (i) the override must be declared in the checked program itself *on a class the same program narrows on — ambient code cannot smuggle one in unseen, since the stubs declare none; (ii) the hole is shared with `tsc`, which also narrows here (a boolean-returning `@@hasInstance` does not disable constructor-based narrowing there; only `is`-predicate returns change it). Cheap repairs exist: reject static `[Symbol.hasInstance]` declarations, or skip narrowing for classes that declare one (the definition is available at the gate).*

impl: `TypeCheckRefine.sk / evalCond (InstanceOf arm)`

impl: `TEnv.sk / addFields (FNComputed), resolveComputedKeyFieldKey`

impl: `TypeCheckExpr.sk / isInstanceofRhs (comment)`

Finding 11 (Instantiation weakness is real: fresh arguments are sealed by later covariant use)**.** *Theorem 9 holds* at the narrowing instant*, and the main paper's* R-INST *lemma correctly delegates the survival of membership under later substitution evolution to its Obligation* Monotone Reads Under Growth *— whose status there is* VIOLATED*, this finding being one of its two witnessed instances: a later covariant read of an unbound candidate does not seal it to* $\top$ *but to the* expected type of the use *(the* $\mathsf{grow}(t, \mathsf{cov})/\mathit{unbound} \to \mathsf{SVClosed}(t)$ *transition of the main paper's Definition "Transitions"), so program* uses *choose the type argument that narrowing had left open — the unsound direction, since no value flow justifies it. Checked witness (accepted at the pin, exit 0).*

`tsc` 5.7 also accepts this program, but through its declared `any` escape hatch: it narrows `unknown` to `Box<any>` and accepts a `string` and a `number` read simultaneously. SKJS's failure mode is different and worse — `any`-free and order-dependent: the first covariant use wins, and a second, differently-typed read is rejected pointing at the first. The witness:

```
class Box<T> { v: T; constructor(v: T) { this.v = v } }
function h(x: unknown): string {
  if (x instanceof Box) {
    const s: string = x.v;   // seals the fresh argument to string
    return s;
  }
  return "no";
}
console.log(h(new Box(42)));  // Node prints: 42
```

h returns `42` at type `string`. The `Array` special case (⊤ argument) is immune, which suggests the repair: instantiate all *`instanceof` type arguments at ⊤ (an explicit top argument — matching tsc in* effect *while avoiding its `any` escape hatch), or mark narrowing-born variables as non-sealable-by-use. Until then, the freshness clause of **L-INST**'s discharge note is* not sufficient*, and the violated instance of the monotonicity obligation should be recorded in the ledger (Section 9).*

impl: `TEnv.sk / makeTArgs; TTypeSubst.sk / Subst::grow (cov/unbound)`

impl: `TypeCheckRefine.sk / evalInstanceof (Array ⊤ special case)`

A-PROTO audit. The pinned stubs omit the surgery surface deliberately: the `Object` statics `create`, `getPrototypeOf`, and `setPrototypeOf` are present only as comments marked "disallowed", and neither `Proxy` nor `Reflect` is declared at all. A checked program therefore cannot express prototype surgery; what remains is the generic unchecked boundary (ambient `declare`d functions, `JSON.parse<T>`) through which any assumption, not specially this one, can be voided — that boundary is priced into **L-TC** already. Status: ASSUMED, adequately enforced for checked code.

impl: `import-stubs/@types/skjs/index.d.ts (Object statics)`

Cross-realm objects. `instanceof` is realm-sensitive: an array (or class instance) from another realm does not have *this* realm's `C.prototype` in its chain, so the operator answers `false`. For soundness of R-INST this is *benign*: the rule narrows only the true branch, and cross-realm values make the test false, landing in the untouched (identity) false branch. Realms are purely a *completeness* phenomenon for `instanceof` — the classical reason `Array.isArray` exists, as the next section exploits. No realm assumption is needed for **L-INST**; we record this so the ledger does not grow a vacuous row.

8 `Array.isArray` and L-ISARR

8.1 The spec side

`Array.isArray` (ECMA-262 §23.1.2.2) is `IsArray` (ECMA-262 §7.2.2): return `true` if the argument is an *Array exotic object* (ECMA-262 §10.4.2: the objects with the special `length`-coupled $[[DefineOwnProperty]]$); if it is a Proxy, recurse on the Proxy's target (throwing a `TypeError` if the proxy has been revoked); otherwise `false`. Three consequences: (i) the test is *cross-realm-safe* — an array from any realm is an Array exotic object, no prototype identity involved — in contrast

to `x instanceof Array`; (ii) it is *Proxy-transparent* — a proxy over an array answers `true`; (iii) on a revoked proxy the *call throws*, so by the main paper's evaluation convention neither branch relation holds and narrowing is vacuously sound for that case. Ground $\mathsf{isArrayV}(o) \triangleq \texttt{IsArray}(\gamma(o))$ (defined, non-throwing case).

8.2 The model side: how `ReadonlyArray` acquires the brand

L-ISARR claims $\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$. Two things could break this: the stub type could demand a *property* real arrays do not have, or a declared member could fail on real arrays. The first danger is real on the surface: the stub spells the brand as an ordinary declaration,

```
declare interface ReadonlyArray<T> extends Iterable<T> {
  readonly __SKJS_isArray: true;
  ...
}
```

and a required field `__SKJS_isArray` would exclude every real runtime array (they carry no such property), making $[\![\mathsf{ReadonlyArray}]\!]$ nearly empty and **L-ISARR** false. The implementation avoids this by construction: `TFields::create` *intercepts* the field named by `kArrayBrandFieldName = "__SKJS_isArray"`, removes it from the field map, and transfers its literal value (`true`/`false`, anything else is a fatal SKJS2051) into the dedicated `TFields.isArray : ?Bool` component — the model's brand isArr, whose membership semantics constrains $\mathsf{isArrayV}$, not props. The brand never exists as an ordinary field of any type built through `TFields::create`, which is the path all class/interface instantiation takes (`TEnv::newObject`). Defaults are set soundly at the same spot: interfaces get $\mathsf{isArr} = \mathsf{None}$ (unknown) and classes get $\mathsf{Some}(\mathsf{false})$ *before* the intercept — so a class inheriting the brand field (e.g. `Array` itself, tuples via `SKJS_Tuple`, or a user `class D extends Array`) still ends at $\mathsf{Some}(\mathsf{true})$, because the inherited brand field is in the collected map and the intercept overrides the default. (Object *literals* take `createFromSorted`, which does not intercept; a user field literally named `__SKJS_isArray` on a literal stays an ordinary — truthfully present — property, which is sound.)

impl: `TType.sk / kArrayBrandFieldName, TFields::create (brand intercept)`

impl: `TEnv.sk / newObject (isArray defaults)`

impl: `import-stubs/@types/skjs/index.d.ts / ReadonlyArray`

8.3 Discharge

Theorem 12 (Discharge of **L-ISARR**)**.** *Assume* ***A-PROXY****: no Proxy exotic object flows into checked code. Then* $\mathsf{isArrayV}(o)$ *implies* $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$.

Proof. Under **A-PROXY**, $\gamma(o)$ is an Array exotic object of some realm. $[\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$ has a single object branch whose constraints we check against the object interface grounded as: $\mathsf{has} = \texttt{HasProperty}$ (prototype chain included — this matches both the `in` operator and property reads) and $\mathsf{get} = [[\mathit{Get}]]$, snapshots per **L-PURE**.

- *Brand*: $\mathsf{isArr} = \mathsf{Some}(\mathsf{true})$ (intercepted as above) requires $\mathsf{isArrayV}(o)$ — the hypothesis.
- *`length`*: required, get-type `number`. Every Array exotic object owns a `length` whose value is a non-negative Number (ECMA-262 §10.4.2).
- *Numeric index* (`readonly [n: number]: T` at $T = \top$): covers numeric-spelling keys not otherwise named; each such property value lies in $[\![\top]\!] = \mathsf{Val}$. Trivially satisfied (and the $\mathsf{set} = \bot$ readonly encoding is invisible to membership, which is get-only).

- *Methods* (`find`, `every`, `some`, `forEach`, `findIndex`, `join`, `map`, `filter`, `flatMap`, `reduce`, `reduceRight`, `slice` — the full declared list at the pin, several classic members being deliberately commented out for variance reasons): each is a required field whose get-type is a single-branch object with a *definite* call signature (interface methods are built through `TObject::createCall`). Membership requires presence and callability of the property value; the behavioral meaning of the signature itself is opaque (bundled into the subtyping surface by the main paper's object semantics). Every listed name is a function-valued property of `Array.prototype` in every realm (ECMA-262 §23.1.3), reached by `get` through the instance's prototype chain; arrays from other realms find their own realm's identical-by-spec `Array.prototype`. (An own property shadowing a method with a non-callable value would falsify this — that is ordinary property mutation, priced into **L-MUT**/**L-PURE** snapshots like every other field statement, not a new caveat.)

- *`[Symbol.iterator]`* (inherited from the `Iterable<T>` base): same argument; the `@@iterator` method of `Array.prototype` exists in every realm, and well-known symbols are *shared across realms* (ECMA-262 §6.1.5.1), so the checker's unique-symbol key denotes the same runtime symbol everywhere.

- *Extensibility*: all other properties of o are unconstrained by the main paper's extensible object semantics; no string index is declared, so no covering constraint arises.

All constraints hold; o inhabits the branch. □

On **A-PROXY**: a Proxy over an array passes `IsArray` but its [[*Get*]] trap may return arbitrary values for `length` or the methods, falsifying membership; hence the assumption. It is enforced for checked code the same way as **A-PROTO**: `Proxy` is absent from the pinned stubs, so no checked program can construct one, and the unchecked boundary is **L-TC**'s. Note the contrast worth keeping in the ledger note: cross-realm values, the classical failure mode of `instanceof Array`, are handled *correctly* here — no realm assumption is required, thanks to `IsArray`'s realm-independence plus the structural (non-nominal) membership semantics.

The mutability note of the main paper (R-IsArray intersects with *Readonly*Array, never mutable `Array`) needs no runtime grounding: it is a precision choice about which *set-types* to add, and set-types do not participate in membership.

9 Status and Ledger Edits

"Here" means in this deep dive.

Item	**Where**
L-TAG	Theorem 2
≈ grounding	Theorem 3, Corollary 4
Truthiness	Theorem 5
ctor⇒call	Theorem 7
A-DDA	§3.1
A-CTOR-CALL	Finding 8
L-INST	Theorem 9

Continued on next page

Item	Where
A-HASINST	Finding 10
A-PROTO	§7
Fresh-arg survival	Finding 11
L-ISARR	Theorem 12
A-PROXY	§8

Registry. The authoritative, current statuses of every item above are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-TAG	Every runtime value carries exactly one tag consistent with its static tag.	PROVED
A-CTOR-CALL	Class-constructor values are typed with call mode None, but constructors implement [[Call]].	VIOLATED
L-INST	$P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type as instantiated at the narrowing site.	VIOLATED
L-ISARR	$\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$.	PROVED
A-PROTO	No prototype surgery / Proxy GetPrototypeOf traps reach checked code (enforced by stub omissions).	ASSUMED
A-DDA	No [[IsHTMLDDA]] object (`document.all`) reaches checked code (vacuous on Node).	ASSUMED
A-PROXY	No Proxy exotic object reaches checked code.	ASSUMED
A-HASINST	No non-default `Symbol.hasInstance` on narrowed `instanceof` RHS classes.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-TC	Checked expressions inhabit their checked types.	main	ASSUMED
L-PURE	Evaluating a condition does not mutate the narrowed locals.	F	VIOLATED
L-MUT	Field writes are governed by set-types; get-only membership is sound for snapshot reads.	main	ASSUMED
OBL-MONOTONE	Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.	E	VIOLATED

Method note (failed attempts). Three discharge attempts failed and became the Findings above; each failure was converted into a checked-and-executed witness rather than a weakened assumption, per the series contract. One further attempt is worth recording as simply *unnecessary*:

we initially looked for a realm assumption for **L-INST** symmetric to the classical `instanceof Array` pitfall, and found that true-branch-only narrowing makes realms a pure completeness concern (§7), so no such row exists.

A Implementation and Specification Anchors

All implementation references are to revision `cf5a6bc7`.

This paper	Anchor
brand name	`TType.sk / kArrayBrandFieldName`
brand intercept	`TType.sk / TFields::create`
brand defaults	`TEnv.sk / newObject`
brand stub spelling	`import-stubs/@types/skjs/index.d.ts`
R-IsArray handler	`TypeCheckRefine.sk / evalIsArray`
array-brand intersection	`TTypeInter.sk / TFields::interIsArray`
R-Inst gate + handler	`TypeCheckRefine.sk / evalCond, evalInstanceof`
instance type arguments	`TEnv.sk / makeTArgs`; `TTypeSubst.sk / Subst::gen`
class-def gate	`TEnv.sk / hasClassDef`
RHS validity (TS2359)	`TypeCheckExpr.sk / isInstanceofRhs`
symbol-keyed class members	`TEnv.sk / addFields (FNComputed)`
static-side class type	`TEnv.sk / newStaticObject`
R-Typeof handler	`TypeCheckRefine.sk / evalTypeofEquality`
truthiness tables	`TType.sk / TPrimValueKind::isFalsy, SType::falsiness`

Specification clauses cited (ECMA-262, 2025-edition numbering; clause names are stable across editions): §6.1 language types; §6.1.5.1 well-known symbols; §6.1.6.1–6.1.6.2 Number/BigInt types (and `Number::equal`, §6.1.6.1.13); §6.1.7.2–6.1.7.3 object internal methods and their invariants; §7.1.2 `ToBoolean`; §7.2.2 `IsArray`; §7.2.3–7.2.4 `IsCallable` and `IsConstructor`; §7.2.10–7.2.12 `SameValue`, `SameValueZero`, and `SameValueNonNumber`; §7.2.16 `IsStrictlyEqual`; §7.3.22 `OrdinaryHasInstance`; §10.2.1 ECMAScript function [[*Call*]] (class-constructor throw); §10.4.1 bound function exotic objects; §10.4.2 Array exotic objects; §10.5.12–10.5.13 Proxy [[*Call*]] and [[*Construct*]]; §13.5.3 `typeof`; §13.10.2 `InstanceofOperator`; §13.11 equality operators; §20.2.3.6 the `@@hasInstance` method of `Function.prototype`; §23.1.2.2 `Array.isArray`; §23.1.3 `Array.prototype` methods; Annex B §B.3.6 [[$IsHTMLDDA$]].

B Witness Protocol

Each Finding's program was (i) written to a temporary file in the repository working tree, (ii) checked with the pinned binary — the `make check-file-in-docker` target, passing `BIN=target/host/dev/skjs` — all three accepted with exit code 0 and no diagnostics — and (iii) type-stripped mechanically and executed with Node to observe the printed value named in the Finding (`[class C]`, `undefined`, `42` respectively). The temporary files were removed afterwards; turning the witnesses into pinned tests is proposed in Section 9 but deliberately not done here (this deep dive changes no file outside `docs/deep-dives/`).